

- Description

This module manages a smart electric scooter system, ensuring that:

1. The system continuously monitors the battery level (in kWh) and calculates the estimated remaining range based on a fixed energy consumption rate (kWh/km).
2. If the battery level falls below a predefined threshold, a low battery warning is triggered.

This system helps users optimize scooter usage by providing real-time battery range estimations and alerts for low battery conditions.

- Specification

module SmartScooterBatteryControl

var currentBattery, batteryCapacity, consumptionRate, lowBatteryThreshold: real

process consumeEnergy(energyUsed: real) returns (currentBattery: real, estimatedRange: real)

functional scenario form:

currentBattery >= lowBatteryThreshold && output "Battery level: " + currentBattery + " kWh, Estimated range: " + estimatedRange + " km.", Energy consumption updated, estimated range calculated normally.

||

currentBattery < lowBatteryThreshold && output "ALERT: Battery level is low (" + currentBattery + " kWh). Please recharge soon. Estimated range: " + estimatedRange + " km.", Low battery alert triggered, estimated range provided.

end process

end module

- Java Code

```
package code;
public class c87_SmartScooter {
    private double batteryCapacity;
    private double currentBattery;
    private double consumptionRate;
    private double lowBatteryThreshold;
    public c87_SmartScooter(double batteryCapacity, double initialBattery, double consumptionRate, double lowBatteryThreshold) {
        this.batteryCapacity = batteryCapacity;
        this.currentBattery = initialBattery;
        this.consumptionRate = consumptionRate;
        this.lowBatteryThreshold = lowBatteryThreshold;
    }
    public void recharge(double newBatteryLevel) {
        if(newBatteryLevel > batteryCapacity) {
            currentBattery = batteryCapacity;
        } else {
```

```

        currentBattery = newBatteryLevel;
    }
    System.out.printf("Battery recharged to: %.2f kWh\n", currentBattery);
}
public void consumeEnergy(double energyUsed) {
    if (energyUsed < 0) {
        System.out.println("Invalid energy consumption value.");
        return;
    }
    currentBattery -= energyUsed;
    if (currentBattery < 0) {
        currentBattery = 0;
    }
    double remainingRange = currentBattery / consumptionRate;
    System.out.printf("Energy Consumed: %.2f kWh, Remaining Battery : %.2f kWh, Estimated Range: %.2f km\n",
        energyUsed, currentBattery, remainingRange);
    if (currentBattery < lowBatteryThreshold) {
        System.out.println("ALERT: Battery level is low! Please recharge soon.");
    }
}
public double getCurrentBattery() {
    return currentBattery;
}

public double getEstimatedRange() {
    return currentBattery / consumptionRate;
}

public static void main(String[] args) {
    c87_SmartScooter scooter = new c87_SmartScooter(2.0, 1.5, 0.1, 0.3);

    // testcase-VT:
    // scooter.consumeEnergy(0.5);
    // scooter.consumeEnergy(0.7);
    // scooter.consumeEnergy(-0.7);
    // scooter.consumeEnergy(1.7);
    // scooter.recharge(2.0);
    // scooter.consumeEnergy(1.8);
    // scooter.recharge(10.0);

```

```

//testcase-FT:
// scooter.consumeEnergy(4.9E-324);
// scooter.consumeEnergy(0.0);
// scooter.consumeEnergy(1.2522728504648997E292);
// scooter.consumeEnergy(3.2071925864657505E294);
// scooter.consumeEnergy(1.7976931348623103E308);
// scooter.consumeEnergy(-7.543261389422266E307);
// scooter.consumeEnergy(7.022238808055902E305);

// testcase-Z3:
// scooter.consumeEnergy(0.5);
// scooter.consumeEnergy(0.7);
// scooter.consumeEnergy(-0.7);
// scooter.consumeEnergy(1.7);
// scooter.recharge(2.0);
// scooter.consumeEnergy(1.8);
// scooter.recharge(10.0);

// testcase-UVT:
scooter.consumeEnergy(0.5);
scooter.consumeEnergy(0.7);
scooter.consumeEnergy(-0.7);
scooter.consumeEnergy(1.7);
scooter.recharge(2.0);
scooter.consumeEnergy(1.8);
scooter.recharge(10.0);
}
}

```

● Coverage report

1. Test case number: 7
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c87_SmartScooter.java	100.0 %	8	0	8
✓ c87_SmartScooter	100.0 %	8	0	8
main(String[])	100.0 %	0	0	0
c87_SmartScooter(double, doub	100.0 %	0	0	0
consumeEnergy(double)	100.0 %	6	0	6
recharge(double)	100.0 %	2	0	2

3. Vibration testing instruction coverage: 93.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	131	21,420	21,551
▼ src	0.6 %	131	21,420	21,551
▼ code	0.6 %	131	21,420	21,551
▼ c87_SmartScooter.java	93.6 %	131	9	140
▼ c87_SmartScooter	93.6 %	131	9	140
main(String[])	100.0 %	30	0	30
c87_SmartScooter(double, double)	100.0 %	15	0	15
consumeEnergy(double)	100.0 %	60	0	60
recharge(double)	100.0 %	26	0	26

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c87_SmartScooter.java	75.0 %	6	2	8
▼ c87_SmartScooter	75.0 %	6	2	8
main(String[])		0	0	0
c87_SmartScooter(double, double)		0	0	0
consumeEnergy(double)	100.0 %	6	0	6

5. Fuzz testing instruction coverage: 75%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	105	21,446	21,551
▼ src	0.5 %	105	21,446	21,551
▼ code	0.5 %	105	21,446	21,551
▼ c87_SmartScooter.java	75.0 %	105	35	140
▼ c87_SmartScooter	75.0 %	105	35	140
main(String[])	100.0 %	30	0	30
c87_SmartScooter(double, double)	100.0 %	15	0	15
consumeEnergy(double)	100.0 %	60	0	60

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c87_SmartScooter.java	100.0 %	8	0	8
▼ c87_SmartScooter	100.0 %	8	0	8
main(String[])		0	0	0
c87_SmartScooter(double, double)		0	0	0
consumeEnergy(double)	100.0 %	6	0	6
recharge(double)	100.0 %	2	0	2

7. Z3 instruction coverage: 93.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	131	21,420	21,551
▼ src	0.6 %	131	21,420	21,551
▼ code	0.6 %	131	21,420	21,551
▼ c87_SmartScooter.java	93.6 %	131	9	140
▼ c87_SmartScooter	93.6 %	131	9	140
main(String[])	100.0 %	30	0	30
c87_SmartScooter(double, double)	100.0 %	15	0	15
consumeEnergy(double)	100.0 %	60	0	60
recharge(double)	100.0 %	26	0	26

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c87_SmartScooter.java	100.0 %	8	0	8
▼ c87_SmartScooter	100.0 %	8	0	8
main(String[])		0	0	0
c87_SmartScooter(double, double)		0	0	0
consumeEnergy(double)	100.0 %	6	0	6
recharge(double)	100.0 %	2	0	2

9. Unimproved V-method instruction coverage: 93.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	131	21,420	21,551
▼ src	0.6 %	131	21,420	21,551
▼ code	0.6 %	131	21,420	21,551
▼ c87_SmartScooter.java	93.6 %	131	9	140
▼ c87_SmartScooter	93.6 %	131	9	140
main(String[])	100.0 %	30	0	30
c87_SmartScooter(double, double)	100.0 %	15	0	15
consumeEnergy(double)	100.0 %	60	0	60
recharge(double)	100.0 %	26	0	26